

```

"""Privacy-Preserving Document Summarizer -
FastAPI backend.
Pipeline: extract text -> detect PII ->
redact -> summarize/Q&A on REDACTED text
only.
Run: uvicorn main:app --reload      then
open http://localhost:8000
"""

import io, os, re, uuid, math
from collections import Counter
from pathlib import Path
from fastapi import FastAPI, UploadFile,
File, Form, HTTPException
from fastapi.middleware.cors import
CORSMiddleware
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

app = FastAPI(title="Privacy-Preserving
Document Summarizer")
app.add_middleware(CORSMiddleware,
allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

# ----- PII detection -----
-----
# (type, regex, group) - group 0 = whole
match, 1 = first capture group
PATTERNS = [
    ("EMAIL",    r"\b[\w.+-]+@[ \w- ]+(?:\.)

```

```

[\w-]+)+\b", 0),
    ("ACCOUNT", r"(?
i)\b(?:a/c|acct|account)(?:\s*
(?:no|number|#))?\.\s*[:\s*
(\d{9,18})\b", 1),
    ("AADHAAR", r"(?!\\d)\\d{4}\\s?\\d{4}\\s?
\\d{4}(?!\\d)", 0),
    ("CARD", r"(?!\\d)(?:\\d[ -]?){13,16}
(?:\\d)", 0),
    ("PAN", r"\\b[A-Z]{5}\\d{4}[A-Z]\\b",
0),
    ("IFSC", r"\\b[A-Z]{4}0[A-Z0-9]
{6}\\b", 0),
    ("PHONE", r"(?!\\d)(?:\\+?91[\\s-]?)?
[6-9]\\d{4}[\\s-]?\\d{5}(?!\\d)", 0),
    ("DOB", r"(?i)\\b(?:dob|date of
birth|born on)\\s*[:\\s*
[/\\-\\.]\\d{1,2}[\\-\\.]\\d{2,4})", 1),
    ("IP", r"\\b(?:\\d{1,3}\\.)
{3}\\d{1,3}\\b", 0),
    ("NAME",
r"\\b(?:Mr|Mrs|Ms|Dr|Shri|Smt)\\.\\s+[A-Z][a-
z]+(?:\\s+[A-Z][a-z]+){0,2}", 0),
]
COMPILED = [(t, re.compile(p), g) for t, p,
g in PATTERNS]

try: # optional NER for names / places
    import spacy
    NLP = spacy.load("en_core_web_sm")
except Exception:
    NLP = None
SPACY_MAP = {"PERSON": "NAME", "GPE":
"LOCATION", "LOC": "LOCATION"}

```

```

def find_pii(text: str):
    found = []
    for typ, rx, g in COMPILED:
        for m in rx.finditer(text):
            found.append({"type": typ,
"start": m.start(g), "end": m.end(g)})
    if NLP:
        for ent in NLP(text[:200000]).ents:
            if ent.label_ in SPACY_MAP:
                found.append({"type":
SPACY_MAP[ent.label_], "start":
ent.start_char, "end": ent.end_char})
        # resolve overlaps: earliest start,
then longest
        found.sort(key=lambda e: (e["start"], -
(e["end"] - e["start"])))
        result, last_end = [], -1
        for e in found:
            if e["start"] >= last_end:
                e["text"] =
text[e["start"]:e["end"]]
                result.append(e)
                last_end = e["end"]
        return result

def redact(text: str, ents):
    counters, seen, out, pos = Counter(),
{}, [], 0
    for e in ents:
        key = (e["type"],
e["text"].lower())

```

```

        if key not in seen:
            counters[e["type"]] += 1
            seen[key] = f"
[{e['type']}]_{counters[e['type']]}"
            e["placeholder"] = seen[key]
            out.append(text[pos:e["start"]])
            out.append(seen[key])
            pos = e["end"]
        out.append(text[pos:])
        return "".join(out)

# ----- Text extraction -----
-----
def extract_text(name: str, data: bytes) ->
str:
    ext = Path(name).suffix.lower()
    if ext == ".pdf":
        from pypdf import PdfReader
        return "\n".join((pg.extract_text()
or "") for pg in
PdfReader(io.BytesIO(data)).pages)
    if ext == ".docx":
        from docx import Document
        return "\n".join(p.text for p in
Document(io.BytesIO(data)).paragraphs)
    return data.decode("utf-8",
errors="ignore")

# ----- Summarizer (local,
extractive) -----
STOP = set("a an the and or but if of to in
on at by for with from as is are was were
be been this that these those it its he she
they we you i not no will shall may can has

```

```

have had do does did than then so
such".split())
def words(s): return [w for w in
re.findall(r"[a-z0-9']+", s.lower()) if w
not in STOP and len(w) > 2]
def sentences(t): return [s.strip() for s
in re.split(r"(?<=[!?])\s+(?=[A-Z\
])|\n{2,}", t) if len(s.strip()) > 20]

def summarize(text: str, n=5):
    sents = sentences(text)
    if not sents: return [], {}
    freq = Counter(w for s in sents for w
in words(s))
    scored = []
    for i, s in enumerate(sents):
        ws = words(s)
        score = sum(freq[w] for w in ws) /
(len(ws) + 5) + (0.3 if i < 2 else 0)
        scored.append((score, i, s))
    top = sorted(sorted(scored,
reverse=True)[:n], key=lambda x: x[1])
    extras = {
        "amounts": sorted(set(re.findall(r"
(?:₹|Rs\.?|INR|\$)\s?[\d,]+(?:\.\d+)?",
text))[:8],
        "dates":
sorted(set(re.findall(r"\b\d{1,2}
[/\-.]\d{1,2}
[/\-.]\d{2,4}\b|\b\d{1,2}\s(?:Jan|Feb|Mar|A
pr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-
z]*\.\?\s\d{4}\b", text))[:8],
    }
    return [s for _, _, s in top], extras

```

```

def llm_summary(redacted: str):
    """Optional: only ever receives
    REDACTED text."""
    key = os.getenv("ANTHROPIC_API_KEY")
    if not key: return None
    try:
        import anthropic
        r = anthropic.Anthropic(api_key=key).messages.create(
            model=os.getenv("LLM_MODEL",
            "claude-sonnet-4-6"), max_tokens=700,
            messages=[{"role": "user",
            "content": "Summarize this document in 5
            bullet points. Placeholders like [NAME_1]
            are redacted values; keep them
            unchanged.\n\n" + redacted[:12000]}])
        return r.content[0].text
    except Exception:
        return None

# ----- Storage: REDACTED text
# only -----
DOCS = {}

@app.post("/api/process")
async def process(file: UploadFile =
File(None), text: str = Form(None)):
    if file is not None:
        raw = extract_text(file.filename,
        await file.read())
    elif text:

```

```

        raw = text
    else:
        raise HTTPException(400, "Upload a
file or paste text")
    if not raw.strip():
        raise HTTPException(400, "No
readable text found")
    ents = find_pii(raw)
    clean = redact(raw, ents)
    points, extras = summarize(clean)
    doc_id = uuid.uuid4().hex[:10]
    DOCS[doc_id] = {"redacted": clean,
"sents": sentences(clean)} # raw text is
never stored
    return {
        "doc_id": doc_id,
        "original": raw,
        "redacted": clean,
        "entities": [{k: e[k] for k in
("type", "start", "end", "placeholder")}
for e in ents],
        "counts": dict(Counter(e["type"]
for e in ents)),
        "summary": {"key_points": points,
"llm": llm_summary(clean), **extras},
        "stats": {"words":
len(raw.split()), "pii_found": len(ents)},
    }

class Ask(BaseModel):
    doc_id: str
    question: str

```

```

@app.post("/api/ask")
def ask(body: Ask):
    doc = DOCS.get(body.doc_id)
    if not doc: raise HTTPException(404,
    "Document not found. Process it again.")
    sents, q = doc["sents"],
    set(words(body.question))
    df = Counter(w for s in sents for w in
    set(words(s)))
    N = len(sents)
    scored = []
    for s in sents:
        ws = set(words(s))
        score = sum(math.log(1 + N / (1 +
    df[w])) for w in q & ws)
        if score > 0: scored.append((score,
    s))
    scored.sort(reverse=True)
    if not scored:
        return {"answer": "I couldn't find
    that in the document.", "sources": []}
    best = [s for _, s in scored[:3]]
    return {"answer": " ".join(best),
    "sources": best}

@app.get("/api/health")
def health(): return {"ok": True, "ner":
    bool(NLP)}

FRONT =
    Path(__file__).resolve().parent.parent /
    "frontend"
    if FRONT.exists():

```

```
    app.mount("/",  
StaticFiles(directory=FRONT, html=True),  
name="frontend")
```